



AULA 3

Técnica aplicada



Mão na massa

Finalmente, você vai ver o jogo funcionando. O código JavaScript vai garantir a interatividade e as respostas aos eventos.

Será preciso tratar uma série de eventos:

- Usuário bate com o martelo (clica com o *mouse* em cima da toupeira ou erra o golpe): deve-se contabilizar positivamente ou negativamente o que aconteceu.
- Usuário abaixa o martelo (clica com o *mouse* e mantém o botão apertado): o *mouse* deve virar um martelo abaixado.
- Usuário levanta o martelo (solta o botão do *mouse*): o *mouse* deve virar um martelo levantado.
- Está na hora de a toupeira subir: de tempos em tempos, é preciso fazer a toupeira subir em um buraco aleatório.
- A toupeira já ficou muito tempo em cima: é preciso fazer a toupeira se esconder no buraco.

Além dos eventos, o jogo também deve poder ser iniciado, atualizar os marcadores e o que mais você pensar que deve acontecer.

Então, é hora de pôr a mão na massa.

Preparando o terreno

Você vai começar criando o diretório **Javascript** e, dentro dele, o arquivo **jogo.js**, para colocar o código JavaScript do jogo.

No arquivo **tabuleiro.html**, que contém o tabuleiro, será criado o *link* para o arquivo *script* na seção **<head>**. O início do arquivo deve ficar assim (observe a linha 18):

<> código

```
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <link rel="stylesheet" href="css/style.css">
7 <!-- Google Fonts -->
8 <link rel="preconnect" href="https://fonts.googleapis.com">
9 <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
10 <link href="https://fonts.googleapis.com/css2?family=Bungee+Spice&display=swap" rel="stylesheet">
11 <!-- Bootstrap -->
12 <link
13   href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
14   rel="stylesheet"
15   integrity="sha384-T3c6CoI16uLRA9tneNEoa7RxnatzjcDSCnG1MXxSR1GAsXEV/Dwvykc29P3122HN"
16   crossorigin="anonymous">
17 <!-- Nosso script -->
18 <script src="javascript/jogo.js"></script>
19
20 <title>Tabuleiro do Jogo</title>
21 </head>
```

Programando o jogo

O que será preciso nesse jogo?

Variáveis

Variáveis globais: as variáveis que armazenam valores que devem ser compartilhados por várias funções e são responsáveis por armazenar dados que valem por todo o programa devem ser criadas de forma global. Todas as outras variáveis devem ser locais. Geralmente, uma variável deve ser local, a menos que represente alguma propriedade global do programa. Uma boa prática é encapsular as variáveis dentro de funções.

Inicialmente, pode-se prever que serão necessárias as seguintes variáveis globais (se você precisar de outras, pode criá-las a qualquer momento; se alguma se mostrar sem necessidade, basta removê-la):

Acertos	↓
Representa a quantidade de acertos do jogador (valor inicial igual a zero, indicando nenhum acerto).	
Perdidos	↓
Representa a quantidade de toupeiras que o jogador deixou passar (valor inicial igual a zero, indicando nenhuma perda).	
Errados	↓
Representa a quantidade de marteladas dadas fora das toupeiras (valor inicial igual a zero, indicando nenhuma martelada errada).	
Intervalo	↓
Intervalo entre o aparecimento das toupeiras (valor inicial igual a 5000, indicando um intervalo de 5000 milissegundos, ou seja, 5 segundos entre os aparecimentos das toupeiras).	
Janela	↓

Janela de tempo que o jogador tem para acertar na toupeira que está fora do buraco (valor inicial de 2000, indicando 2000 milissegundos, ou 2 segundos, que será o tempo que a toupeira ficará fora do buraco).

O *script* com as variáveis locais criadas e inicializadas:

código

```
1  /** quantidade de acertos */
2  var acertos;
3  acertos = 0;
4
5  /** quantidade de toupeiras perdidas */
6  var perdidos = 0;
7
8  /** quantidade de marteladas erradas */
9  var errados = 0;
10
11 /** tempo entre cada toupeira sair do buraco */
12 var intervalo = 5000;
13
14 /** tempo que a toupeira fica fora do buraco */
15 var janela = 2000;
```

Dois exemplos de como criar e inicializar uma variável. Nas linhas 2 e 3, a variável **acertos** foi criada e inicializada, respectivamente. Na linha 6, a variável **perdidos** foi criada e inicializada. O resultado é o mesmo.

Eventos

Eventos são acontecimentos ou ações que ocorrem durante a vida de uma página e, normalmente, estão associados a um elemento HTML.

A propriedade (atributo) que armazena o evento tem o mesmo nome deste, apenas começando por **on**. Por exemplo, o evento **click**, referente ao clique do *mouse*, é armazenado no atributo **onclick**.

- **load (onload):** o evento **load**, associado ao atributo **onload**, acontece quando um elemento HTML termina de ser carregado pelo navegador. Será usado o evento **load** na página para iniciar o *script*. Essa é uma boa prática, pois evita que o programa comece antes de todos os elementos HTML serem carregados. Se o *script* tentar acessar algum elemento HTML que ainda não está disponível, vai disparar uma exceção (erro).
- **click (onclick):** o evento **click**, associado ao atributo **onclick**, acontece quando o usuário clica com o botão esquerdo do *mouse* em um elemento HTML. Esse evento será usado em duas situações distintas: (1) associado ao botão Start, para começar o jogo quando o usuário clicar no botão; e (2) associado a cada um dos cinco buracos do gramado, para realizar a contabilidade do jogo.
- **mousedown (onmousedown):** o evento **mousedown**, associado ao atributo **onmousedown**, acontece quando o usuário mantém pressionado o botão esquerdo do *mouse*. Sempre que o cursor do *mouse* estiver sobre o gramado e o usuário apertar o botão esquerdo, mantendo-o pressionado, o cursor será alterado para um martelo abaixado, simbolizando uma martelada.
- **mouseup (onmouseup):** o evento **mouseup**, associado ao atributo **onmouseup**, acontece quando o usuário solta o botão esquerdo do *mouse*, depois de tê-lo pressionado. Quando o cursor do *mouse* estiver sobre o gramado e o usuário liberar o botão esquerdo, devemos trocar o cursor para um martelo levantado, indicando que o usuário levantou o martelo.

Associando eventos a elementos HTML

Os eventos podem ser associados a elementos HTML de duas formas: *inline* ou por JavaScript (novamente, este foi eleito aqui como favorito).

→ Inline

Você pode usar os atributos HTML **on** para associar eventos diretamente a elementos HTML. No exemplo a seguir, a função **minhaFuncao** é executada ao clicar com o botão esquerdo do *mouse* no botão:



Embora essa forma de associar um evento a um elemento HTML seja a mais fácil de entender e implementar, tem grandes desvantagens, como dificuldades de manutenção e falta de separação de estrutura (HTML), estilo (CSS) e comportamento (JavaScript).

→ JavaScript

Uma maneira mais moderna de associar eventos é usar o método **addEventListener**. Esse método permite adicionar um *listener* a um elemento HTML de forma mais flexível.

Por exemplo, pode-se usar o seguinte trecho de código JavaScript para adicionar um evento **click** a um botão identificado por **idButton**:



De forma mais compacta, pode-se também criar uma função anônima ao associar o evento:

```
document.getElementById("idButton").onclick = function() {
  alert("O botão foi clicado!");
};
```

Tratando os eventos

Agora, você vai associar funções que irão tratar os eventos aos eventos que serão tratados em cada objeto ou elemento HTML.

Para adicionar um *listener* a um elemento HTML, deve-se trazer o elemento HTML para o mundo JavaScript. O objeto **document** representa o documento HTML com todos os seus elementos. Esse objeto tem o método **getElementByld**, que, como o nome sugere, pega um elemento HTML pela sua identificação.

O exemplo a seguir associa um botão HTML, identificado por **idButton**, a uma variável JavaScript chamada **botão**:

```
var botao = document.getElementById('idButton');
```

Para adicionar um *listener* de evento de clique de *mouse* chamado **trataClick** a esse botão, pode-se usar o método **addEventListener**:

```
botao.addEventListener('click', trataClick);
```

Como a variável **botao** vale exatamente o que está do lado direito da atribuição, **pode-se escrever o mesmo código, de forma mais compacta, em uma linha, substituindo a variável botao pelo seu valor**:

```
document.getElementById('idButton').addEventListener('click', trataClick);
```

→ Carga da página

O evento gerado pelo término da transferência da página do servidor para o cliente pode ser usado para implementar a configuração inicial de todas as atividades da página. No caso do jogo *Whac-A-Mole*, será aproveitado o evento **load** da janela (**window**), associando os outros eventos aos elementos HTML recém-criados.

```
15 onload = function () {
16   document.getElementById('start').addEventListener('click', start);
17   document.getElementById('idGrado').addEventListener('mousedown', marteloBaixo);
18   document.getElementById('idGrado').addEventListener('mouseup', marteloCima);
19   document.getElementById('buraco0').addEventListener('click', martelada);
20   document.getElementById('buraco1').addEventListener('click', martelada);
21   document.getElementById('buraco2').addEventListener('click', martelada);
22   document.getElementById('buraco3').addEventListener('click', martelada);
23   document.getElementById('buraco4').addEventListener('click', martelada);
24 };
```

A linha 15 associa a função anônima descrita logo a seguir ao evento **load** da janela.



Fique ligado

Em JavaScript, o objeto que representa a janela do navegador se chama **window**, e é o objeto *default* (padrão). Isso significa que não é necessário escrever o nome do objeto, ou seja, a linha 15 é equivalente a escrever:

```
15 window.onload = function () {
```

As linhas 16 a 23 associam eventos gerados em elementos HTML obtidos por meio de sua identificação a funções que serão criadas. Por exemplo, a linha 17 associa o evento de passar o *mouse* por cima do elemento (**mousedown**) identificado por **idGramado** à função **marteloBaixo**, que ainda será criada.

```
document.getElementById('idGramado').addEventListener('mousedown',marteloBaixo);
```

Identificação do elemento que vai sofrer a ação.

Tipo (nome) do evento.

Nome da função que vai tratar o evento.

→ Iniciando o jogo

A função *start*, que irá tratar o evento de clique do botão de mesma identificação, deve:

- remover o evento do botão Start para que o usuário não possa inicializar o jogo várias vezes;
- desabilitar o botão Start;
- subir a toupeira pela primeira vez, chamando uma função para realizar essa tarefa.

Código HTML do botão:

```
<button id="start" class="btn btn-light">Start</button>
```

Código JavaScript para tratar o evento clique no botão:

```
22 function start() {
23   var botao = document.getElementById('start')
24
25   botao.removeEventListener('click', start);
26   botao.disabled = true;
27   sobeToupeira();
28 }
```

- **Linha 23:** traz o botão HTML para JavaScript.
- **Linha 25:** remove o evento de clique do *mouse*.
- **Linha 26:** desabilita o botão.
- **Linha 27:** chama a função que sobe uma toupeira (a fazer em seguida).

→ Subindo a toupeira

A função que irá mostrar a toupeira em um buraco deve escolher aleatoriamente um dos cinco buracos e colocar a imagem do buraco com a toupeira no lugar da imagem do buraco sem a toupeira.

Logo depois, a função deve ativar dois temporizadores (*timers*): um para remover a toupeira do buraco e o outro para subir a próxima toupeira. A função **setTimeout** permite criar um *timer* que, ao chegar ao fim, irá chamar a função especificada como primeiro parâmetro. O tempo em milissegundos é informado como segundo parâmetro. Opcionalmente, podem-se incluir parâmetros para a função a ser executada no momento do *timeout*. Os parâmetros devem ser informados após o valor do temporizador. Essa função retorna um objeto **timer** que pode ser utilizado para interromper o temporizador, se necessário.

Código da função para subir a toupeira:

```
36 function sobeToupeira() {
37   var buraco = Math.floor(Math.random() * 5);
38   var objBuraco = document.getElementById("buraco" + buraco);
39   objBuraco.src = "images/hole-mole.png";
40   timer = setTimeout(tiraToupeira, janela, buraco);
41   setTimeout(sobeToupeira, intervalo);
42 }
```

- **Linha 37:** o método **random** do objeto **Math** sorteia um número fracionário aleatório entre 0 (*inclusive*) e 1 (*exclusive*). Ao multiplicar por 5, o intervalo passa a ser um valor fracionário aleatório entre 0 (*inclusive*) e 5 (*exclusive*). O método **floor** do objeto **Math** trunca o valor, ignorando a parte fracionária, resultando em um valor inteiro no intervalo entre 0 (*inclusive*) e 5 (*exclusive*), ou seja, entre 0 (*inclusive*) e 4 (*inclusive*). Esse valor é atribuído à variável local chamada **buraco**, que irá representar o número do buraco sorteado para a toupeira aparecer.
- **Linha 38:** a expressão '**buraco**' + **buraco** concatena o texto '**buraco**' com o valor da variável **buraco**. Por exemplo, se o valor aleatório obtido na linha 37 for 3, o resultado dessa expressão será '**buraco3**'. O método **getElementById** pega o elemento HTML identificado pela expressão resultante e o atribui à variável JavaScript **objBuraco**. Esse é o buraco onde a toupeira irá aparecer. Lembre-se: esse objeto é um elemento do tipo ****.

```
31 
32 
33 
34 
35 
```

- **Linha 39:** troca a imagem usando o atributo **src**. A imagem anterior era de um buraco sem a toupeira. Agora, troca-se essa imagem por uma com a toupeira. Essa imagem está no diretório **images**. Um detalhe importante: as duas imagens devem ser do mesmo tamanho para evitar que o gramado fique deformado ou torto.
- **Linha 40:** cria um temporizador que irá chamar a função **tiraToupeira** (primeiro parâmetro) depois do tempo em milissegundos especificado pela variável **janela** (segundo parâmetro). O terceiro parâmetro, que é o número do buraco, será usado como primeiro parâmetro pela função **setTimeout** ao chamar a função **tiraToupeira**, ou seja, quando se implementa a função **tiraToupeira**, tem-se que receber, como primeiro parâmetro, o número do buraco onde a toupeira está aparecendo. O objeto do tipo *timer* que a função **setTimeout** retorna será armazenado na variável global **timer** para poder cancelar ou antecipar a remoção da toupeira do buraco na

função que irá tratar a martelada caso o usuário acerte uma toupeira.

Como essa variável não havia sido criada anteriormente, será modificada a lista de variáveis globais, que vai ficar como mostrado a seguir (veja a linha 18):

```
1  /** quantidade de acertos */
2  var acertos;
3  acertos = 0;
4
5  /** quantidade de toupeiras perdidas */
6  var perdidos = 0;
7
8  /** quantidade de marteladas erradas */
9  var errados = 0;
10
11 /** tempo entre cada toupeira sair do buraco */
12 var intervalo = 5000;
13
14 /** tempo que a toupeira fica fora do buraco */
15 var janela = 2000;
16
17 /** timer que controla o tempo da toupeira fora do buraco */
18 var timer = null;
```

- **Linha 41:** independentemente do que aconteça, se o jogador acertou ou não a toupeira, deve-se subir uma nova toupeira depois que o tempo em milissegundos especificado pela variável **intervalo** tiver passado.

Importante: o valor da variável **janela**, relativo ao tempo que a toupeira deve ficar fora do buraco, deve ser menor do que o valor da variável **intervalo**, que representa o intervalo entre duas subidas consecutivas de toupeiras nos buracos.

→ Removendo a toupeira

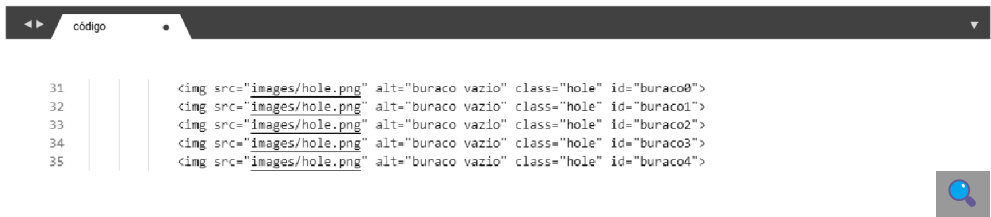
A função **tiraToupeira** é chamada quando o evento de tempo de toupeira aparecendo no buraco acabar. Essa função recebe como parâmetro o número do buraco onde a toupeira está para que não se precise remover a toupeira de todos os buracos (lembre-se de que se deve fazer o *script* rodar rápido porque não se sabe nada sobre a máquina do usuário que irá usar esse jogo).

A função **tiraToupeira** deve remover a toupeira do buraco informado, trocando sua imagem por uma com apenas o buraco. Se a toupeira foi removida por essa função, significa que o jogador não acertou a toupeira com o martelo, ou seja, deve-se incrementar a variável **perdidos**. Além disso, já que a pontuação mudou, será chamada a função **mostraPontuacao** para mostrar a nova pontuação do jogador.

Código da função para baixar a toupeira:

```
51 function tiraToupeira(buraco) {
52     var objBuraco = document.getElementById('buraco' + buraco);
53     objBuraco.src = 'images/ho1e.png';
54     perdidos++;
55     mostraPontuacao();
56 }
```

- **Linha 51:** a função **tiraToupeira** recebe o parâmetro **buraco** representando o número do buraco onde a toupeira está.
- **Linha 52:** a expressão '**buraco**' + **buraco** concatena o texto '**buraco**' com o valor da variável **buraco**. Por exemplo, se o valor da variável for 3, o resultado dessa expressão será '**buraco3**'. O método **getElementById** pega o elemento HTML identificado pela expressão resultante e o atribui à variável JavaScript **objBuraco**. Esse é o buraco onde a toupeira está. Lembre-se: esse objeto é um elemento do tipo ****.



- **Linha 53:** troca a imagem usando o atributo **src**. A imagem anterior era de um buraco com a toupeira. Agora, essa imagem foi trocada por uma sem a toupeira. Essa imagem está no diretório **imagens**. Um detalhe importante: as duas imagens devem ser do mesmo tamanho para evitar que o gramado fique deformado ou torto.
- **Linha 54:** incrementa a quantidade de vezes que uma toupeira apareceu e o jogador não a acertou com o martelo. Escrever **perdidos++** é o mesmo que escrever **perdidos = perdidos + 1**. Os programadores JavaScript, assim como os programadores Java, C, C++, etc., gostam de usar essa forma por ser mais compacta e, com o tempo, você achará mais fácil de ler também.
- **Linha 55:** chama a função **mostraPontuacao**, que atualiza o placar com a pontuação do jogador.

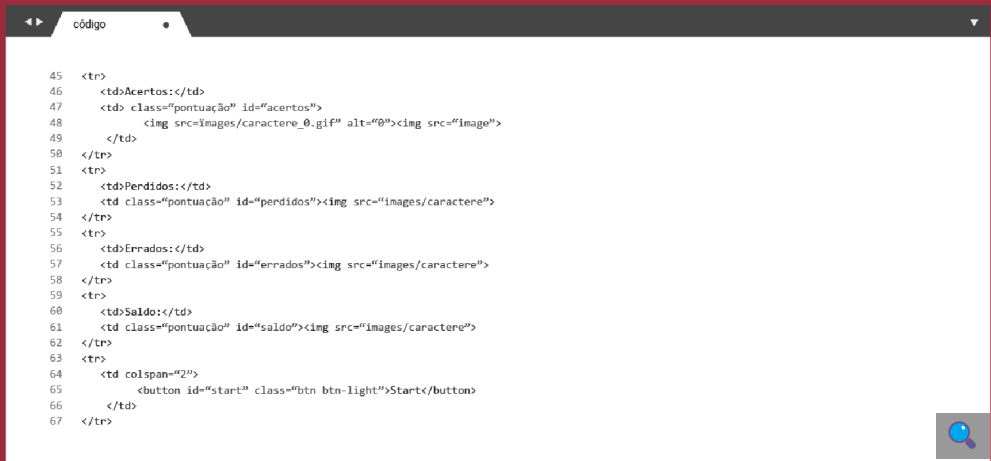
→ Mostrando a pontuação – parte 1

A função **mostraPontuacao** exibe a pontuação do jogador chamando quatro vezes a função **mostraPontuacaoDe**, cada uma para um tipo de valor a ser exibido: marteladas certas, toupeiras que não foram acertadas, marteladas erradas e pontuação total. Como a pontuação total não deve ser mostrada com valores negativos, pois o *display* do placar não tem números negativos, então sempre será exibido o maior valor entre zero e a pontuação do usuário. Se a pontuação for menor do que zero, o valor exibido será zero.



Fique ligado

Veja que cada um dos quatro valores do *display* é composto por três imagens colocadas dentro de uma célula de tabela. As imagens são filhas de células identificadas pelos seguintes valores de seus respectivos atributos *id*: **acertos**, **perdidos**, **errados** e **saldo**. A centena é o primeiro filho, a dezena é o primeiro irmão do primeiro filho e a unidade é o primeiro irmão da dezena. Você vai se lembrar disso quando implementarmos a função **mostraPontuacaoDe**?



Código da função **mostraPontuacao**:

```

código
62 function mostraPontuacao() {
63   mostraPontuacaoDe('acertos', acertos);
64   mostraPontuacaoDe('perdidos', perdidos);
65   mostraPontuacaoDe('errados', errados);
66   mostraPontuacaoDe('saldo', Math.max(acertos - perdidos - errados, 0));
67 }

```

- **Linhas 63, 64 e 65:** exibem os valores de acertos, perdidos e errados, respectivamente.
- **Linha 66:** calcula a pontuação do jogador somando os acertos e subtraindo os pontos perdidos.

→ **Mostrando a pontuação – parte 2**

A função **mostraPontuacaoDe** efetivamente mostra a pontuação, mas de forma genérica. Essa função recebe como parâmetro o lugar onde a pontuação deve ser exibida – pai das imagens – e seu valor. O valor da pontuação não pode exceder três algarismos.

Antes de entrar na parte de programação, você vai entender como o valor é exibido. No diretório com as imagens, existem 10 imagens do tipo GIF, com chamadas **caractere_0.gif**, **caractere_1.gif** até **caractere_9.gif**, representando os números de 0 até 9 no formato de um antigo *display* de dezesseis segmentos. As imagens foram extraídas de um GIF animado da Wikipédia (2020).



Por exemplo, para criar o valor 875, devemos usar as imagens **caractere_8.gif**, **caractere_7.gif** e **caractere_5.gif**.

Veja que as partes inicial e final do nome do arquivo são sempre as mesmas, mudando apenas o número da imagem.

Código da função **mostraPontuacaoDe** que efetivamente mostra a pontuação:

```

código
75 function mostraPontuacaoDe(display, valor) {
76   // pega as imagens
77   let objCentena = document.getElementById(display).firstChild;
78   let objDezena = centena.nextSibling;
79   let objUnidade = dezena.nextSibling;
80
81   // calcula o valor de cada algarismo
82   let centena = parseInt(valor/100);
83   let dezena = parseInt((valor/10)%10);
84   let unidade = (valor % 10);
85
86   // muda a imagem e o valor do atributo para ledor de tela
87   objCentena.src = 'images/caractere_' + centena + '.gif';
88   objCentena.alt = centena;
89   objDezena.src = 'images/caractere_' + dezena + '.gif';
90   objDezena.alt = dezena;
91   objUnidade.src = 'images/caractere_' + unidade + '.gif';
92   objUnidade.alt = unidade;
93 }

```

- **Linha 75:** recebe como parâmetros a identificação do pai do trio de imagens e o valor a ser colocado como figuras, respectivamente.
- **Linha 77:** pega o objeto **** referente à centena. Esse objeto é o primeiro filho (**firstChild**) da célula identificada pelo valor da variável **display**.
- **Linha 78:** pega o objeto **** referente à dezena. Esse objeto é o próximo irmão do objeto **** da centena.
- **Linha 79:** pega o objeto **** referente à unidade. Esse objeto é o próximo irmão do objeto **** da dezena.

- **Linha 82:** calcula o algarismo da centena do valor a ser exibido. O algarismo da centena é igual ao valor dividido por 100. Como o resultado dessa conta pode dar um valor fracionário, será usada a função **parseInt**, que retorna um valor inteiro relativo ao parâmetro passado a ela. Por exemplo: se o valor a ser exibido for 875, ao dividir por 100, obteremos 8,75 (8.75 no computador). O valor inteiro referente a esse número será 8, que é o algarismo da centena.

O operador / calcula a divisão do valor à esquerda pelo valor à direita.

O operador % retorna o resto da divisão do número à esquerda pelo número à direita.

- **Linha 83:** calcula o algarismo da dezena do valor a ser exibido. O algarismo da dezena é igual valor inteiro do resto da divisão por 10 do valor dividido por 10. Por exemplo: se o valor for 875, ao dividir por 10, obtem-se 87,5 (87.5). O resto da divisão por 10 será 7,5 (7.5), e o valor inteiro equivalente será 7, que é o algarismo da dezena a ser exibida.
- **Linha 84:** calcula o algarismo da unidade do valor a ser exibido. O algarismo da unidade é igual ao resto da divisão por 10 do valor total. Por exemplo: se o valor total for 875, o resto da divisão por 10 será 5, que é o valor da unidade a ser exibida.
- **Linhas 87, 89 e 91:** trocam a imagem por uma nova, referente ao valor da centena, dezena e unidade, respectivamente, concatenando o texto '**caractere_**' com o valor da centena, dezena ou unidade e o texto '**.gif**' com a extensão do arquivo.
- **Linhas 88, 90 e 92:** trocam o valor do atributo **alt** da tag **** de acordo com a imagem que está sendo exibida para o ledor de tela.

→ Abaixando o martelo

A função **marteloBaixo** é chamada sempre que o usuário mantiver o botão esquerdo do *mouse* apertado quando o cursor estiver sobre o elemento que representa o gramado. A função troca o formato do cursor do *mouse* para um martelo abaixado.

Código HTML do gramado:

```
30      <div class="gramado" id="idGramado">
31          
32          
33          
34          
35          
36      </div>
```

Código da função **marteloBaixo**:

```
109  function marteloBaixo() {
110      document.getElementById('idGramado').style.cursor = 'url(images/hammerDown.png), default';
111  }
```

- **Linha 110:** muda o estilo do cursor do elemento identificado como gramado. A função **url** cria um URL a partir de seu parâmetro. Como pode acontecer algum imprevisto e a carga da imagem falhar, devemos sempre informar uma lista de imagens

como *fallback*. No caso, foi especificado que o *fallback* seria o cursor-padrão (*default*).

→ Subindo o martelo

A função **marteloCima** é chamada sempre que o usuário soltar o botão esquerdo do *mouse* quando o cursor estiver sobre o elemento que representa o gramado. A função troca o formato do cursor do *mouse* para um martelo levantado.

Código HTML do gramado:

```
30 | <div class="gramado" id="idGramado">
31 |   
32 |   
33 |   
34 |   
35 |   
36 | </div>
```

Código da função **marteloCima**:

```
116 function marteloCima() {
117 |   document.getElementById('idGramado').style.cursor = 'url(images/hammer.png), default';
118 }
```

- **Linha 117:** muda o estilo do cursor do elemento identificado como gramado. A função **url** cria um URL a partir de seu parâmetro. Como pode acontecer algum imprevisto e a carga da imagem falhar, deve-se sempre informar uma lista de imagens como *fallback*. No caso, foi especificado que o *fallback* seria o cursor-padrão (*default*).

→ Dando martelada

Toda função registrada pelo método **addEventListener** como *listener* de um evento pode receber um objeto como parâmetro representando o evento. Entre os diversos atributos desse objeto, o que interessa agora é o **target**, que representa o objeto (elemento HTML em JavaScript) em que o evento ocorreu. Como todos os *listeners* do evento associado à martelada estão em elementos do tipo **** representando os buracos das toupeiras, pode-se inspecionar seu atributo **src** para verificar se o valor **hole-mole**, que é parte do nome do arquivo com a imagem da toupeira aparecendo fora do buraco, faz parte de seu valor:

- (1) Se fizer parte, significa que o jogador acertou a martelada na toupeira. Logo, deve ganhar ponto por isso (lembre-se de que esse jogo é do século passado). Além disso, deve-se tirar a toupeira do buraco e remover o *timer* que faz isso (afinal, ela não está mais no buraco).
- (2) Se não fizer parte, significa que o jogador errou a martelada. Logo, deve-se incrementar o contador de marteladas erradas.

Em ambos os casos, o placar mudou. Então, deve-se chamar a função que atualiza o placar.

Código da função **martelada**:

```
127 function martelada(evento) {
128     if (evento.target.src.includes('hole-mole')) {
129         // acertou
130         acertos++;
131         evento.target.src = 'images/hole.png';
132         clearTimeout(timer);
133     }
134     else {
135         // errou
136         errados++;
137     }
138     mostraPontuacao();
139 }
```

- **Linha 127:** a função recebe o objeto evento representando o evento que acabou de ocorrer.
- **Linha 128:** o objeto evento tem o atributo **target** que representa o objeto em que o evento ocorreu. Nesse caso, um elemento do tipo ****. Esse elemento tem o atributo **src**, que contém o nome do arquivo com a imagem. Se **hole-mole** estiver incluído no nome do arquivo, a toupeira está nesse buraco.
- **Linha 130:** incrementa a quantidade de acertos. (Lembrou que **acertos++** é equivalente a **acertos = acertos +1**?)
- **Linha 131:** remove a toupeira do buraco, trocando a imagem para uma de buraco sem toupeira.
- **Linha 132:** remove o *timer* que tira a toupeira do buraco, pois ela já saiu de lá e não precisa ser removida novamente. Se a toupeira for removida pela função associada a esse *timer*, o contador irá apresentar um erro, que não aconteceu, do jogador.
- **Linha 134:** se o jogador não acertou a martelada no buraco com a imagem da toupeira...
- **Linha 136:** o contador de marteladas erradas é incrementado.
- **Linha 138:** chamada da função que atualiza o placar.

Conclusão

Agora, o *site* do jogo **Whac-A-Mole** está pronto. Teste-o exaustivamente, para ver se está tudo funcionando.



Mantendo o foco

Lembre-se: todo esse código-fonte está disponível para você no repositório.

[Clique aqui.](#)

Algumas coisas no *site* poderiam ficar melhores. Verifique o que você acha que pode ser aprimorado e... mãos à obra!